

NOSQL REPRESENTATION OF PRODUCT CATALOG

1. Introduction

The E-Commerce Order Management System uses a relational database containing information about products, categories, sellers, customers, orders, payments, and shipping. The Product entity contains details such as Product ID, Product Name, Category, Seller, Price, Stock Quantity, and Product Status.

A NoSQL representation can store the product catalog as **JSON documents** and **key-value pairs**. This provides a flexible way to represent product information without using traditional relational tables.

2. JSON Document Representation

Each product can be represented as an individual JSON document.

Product 1

```
{
  "product_id": 401,
  "product_name": "Wireless Mouse",
  "category": {
    "category_id": 301,
    "category_name": "Electronics"
  },
  "seller": {
    "seller_id": 201,
    "seller_name": "Tech World"
  },
  "price": 799.00,
  "stock_quantity": 50,
  "product_status": "Available"
}
```

Product 2

```
{  
  "product_id": 402,  
  "product_name": "Bluetooth Headphones",  
  "category": {  
    "category_id": 301,  
    "category_name": "Electronics"  
  },  
  "seller": {  
    "seller_id": 201,  
    "seller_name": "Tech World"  
  },  
  "price": 2499.00,  
  "stock_quantity": 30,  
  "product_status": "Available"  
}
```

Product 3

```
{  
  "product_id": 403,  
  "product_name": "Cotton T-Shirt",  
  "category": {  
    "category_id": 302,  
    "category_name": "Clothing"  
  },  
  "seller": {  
    "seller_id": 202,  
    "seller_name": "Fashion Hub"  
  },  
}
```

```
"price": 599.00,  
"stock_quantity": 100,  
"product_status": "Available"  
}
```

Product 4

```
{  
  "product_id": 404,  
  "product_name": "Denim Jeans",  
  "category": {  
    "category_id": 302,  
    "category_name": "Clothing"  
  },  
  "seller": {  
    "seller_id": 202,  
    "seller_name": "Fashion Hub"  
  },  
  "price": 1499.00,  
  "stock_quantity": 40,  
  "product_status": "Available"  
}
```

Product 5

```
{  
  "product_id": 405,  
  "product_name": "Database Management Book",  
  "category": {  
    "category_id": 303,  
    "category_name": "Books"  
  },  
}
```

```
"seller": {  
  "seller_id": 203,  
  "seller_name": "Book Store"  
},  
"price": 850.00,  
"stock_quantity": 25,  
"product_status": "Available"  
}
```

These examples use the same product, category, and seller information defined in the project design.

3. Product Catalog as a JSON Collection

The products can also be stored together as a collection:

```
[  
  {  
    "product_id": 401,  
    "product_name": "Wireless Mouse",  
    "category": {  
      "category_id": 301,  
      "category_name": "Electronics"  
    },  
    "seller": {  
      "seller_id": 201,  
      "seller_name": "Tech World"  
    },  
    "price": 799.00,  
    "stock_quantity": 50,  
    "product_status": "Available"  
  }  
]
```

```
},  
{  
  "product_id": 402,  
  "product_name": "Bluetooth Headphones",  
  "category": {  
    "category_id": 301,  
    "category_name": "Electronics"  
  },  
  "seller": {  
    "seller_id": 201,  
    "seller_name": "Tech World"  
  },  
  "price": 2499.00,  
  "stock_quantity": 30,  
  "product_status": "Available"  
},  
{  
  "product_id": 403,  
  "product_name": "Cotton T-Shirt",  
  "category": {  
    "category_id": 302,  
    "category_name": "Clothing"  
  },  
  "seller": {  
    "seller_id": 202,  
    "seller_name": "Fashion Hub"  
  },  
  "price": 599.00,
```

```
"stock_quantity": 100,
"product_status": "Available"
},
{
"product_id": 404,
"product_name": "Denim Jeans",
"category": {
"category_id": 302,
"category_name": "Clothing"
},
"seller": {
"seller_id": 202,
"seller_name": "Fashion Hub"
},
"price": 1499.00,
"stock_quantity": 40,
"product_status": "Available"
}
]
```

4. Key-Value Representation

In a key-value model, each product attribute is represented as a **key-value pair**.

Product 401

Product:401

product_id = 401

product_name = Wireless Mouse

category_id = 301

category_name = Electronics

seller_id = 201

seller_name = Tech World

price = 799.00

stock_quantity = 50

product_status = Available

Product 402

Product:402

product_id = 402

product_name = Bluetooth Headphones

category_id = 301

category_name = Electronics

seller_id = 201

seller_name = Tech World

price = 2499.00

stock_quantity = 30

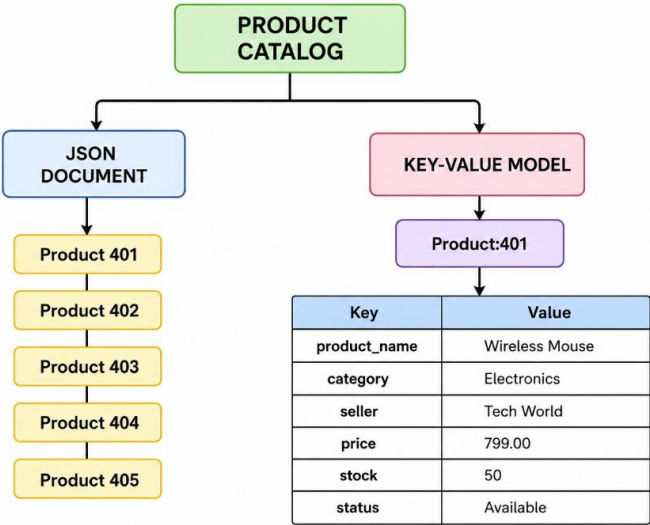
product_status = Available

5. Key-Value Table

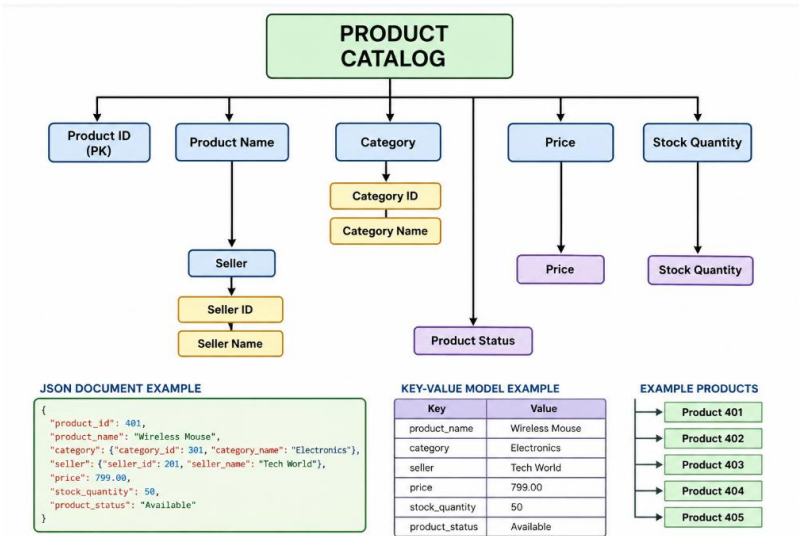
Key	Value
product_id	401
product_name	Wireless Mouse
category_id	301
category_name	Electronics
seller_id	201
seller_name	Tech World

Key	Value
price	799.00
stock_quantity	50
product_status	Available

6. NoSQL Product Catalog Structure Diagram



7. JSON Document Structure



8. Comparison with Relational Model

Relational Database	NoSQL Representation
Product table	JSON Product Document
Product_ID	product_id
Product_Name	product_name
Category_ID (FK)	Embedded category
Seller_ID (FK)	Embedded seller
Price	price
Stock_Quantity	stock_quantity
Product_Status	product_status
Rows and columns	JSON key-value pairs
Foreign-key relationships	Embedded/nested data

In the relational design, Product uses Category_ID and Seller_ID as foreign keys to connect Product with Category and Seller. In the NoSQL representation above, the category and seller information is embedded directly inside the product document.

9. Advantages of the NoSQL Representation

The JSON-based representation provides a flexible structure for product information. Category and seller details can be stored together with the product, making the document self-contained. New product attributes can also be added without redesigning a fixed table structure.

10. Conclusion

The Product Catalog of the E-Commerce Order Management System can be represented using both **JSON documents** and a **key-value model**. JSON documents organize all relevant product information in a single structured document, while the key-value model represents each attribute as a key and its corresponding value. This

demonstrates how the relational Product design can be represented using a NoSQL approach.